

NO. 885e80783xuv8079 | 2026-05-24 13:49:17

- 题目：面向亿门级FPGA电路的划分算法研究
- 作者：张驭龙
- 检测所属单位：福州大学

📄 论文字符数：37377 📄 论文页数：58 📊 表格数量：9 🖼️ 图片数量：12

检测结果



13.6%

全文总相似比 (复写率+自引率+他引率+专业术语)

相似结果详情

9.62%	0.0%	3.78%	0.2%
复写率	自引率	他引率	专业术语

其他指标

去除引用本人文献相似率：13.6% 去除专业术语相似率：13.4% 自写率：86.4%

典型相似文章：无

检测范围 | 1989-01-01 ~ 2026-05-24

- | | | |
|-----------------|------------------|-------------------|
| ■ 中文科技期刊论文全文数据库 | ■ 博士/硕士学位论文全文数据库 | ■ 外文特色文献数据全库 |
| ■ 中文主要报纸全文数据库 | ■ 中国专利特色数据库 | ■ 中国主要会议论文特色数据库 |
| ■ 古籍文献/图书资源 | ■ IPUB原创作品 | ■ 互联网数据资源/互联网文档资源 |
| ■ 港澳台文献资源 | ■ 年鉴资源 | ■ 维普优先出版论文全文数据库 |
| ■ 高校自建资源库 | | |

相似片段

相似片段：

98	75	23
总相似片段	相似片段	引用片段

检测来源：

期刊 36	综合 3	外文 0
博硕 26	互联网 24	自建库 9

引用文献汇总（当前只展示10条数据,全部详情请查看片段对照报告）

引用文献来源：13

序号	引用文献	引用字符数	引用率	来源
1	面向FPGA的布局与布线技术研究综述 田春生;陈雷;王源;王硕;周婧;张瑶伟;庞永江;周冲;马筱婧;杜忠 - 电子学报 - 2022	537	3.54%	期刊
2	单晶炉液位检测的曲面镜拟合方法研究 何恒盛 - 2016	77	0.51%	博硕
3	SAGA-SVM模型在枯水期月径流预测中的应用 毛宗波 - 珠江现代建设 - 2017	66	0.44%	期刊
4	大学生毕业论文-148833	63	0.42%	互联网
5	混合智能算法在半导体生产线生产计划中的应用研究 李兆佳 - 2009	60	0.40%	博硕
6	031102431_陈航迪_智能算法在TSP问题的研究与应用 2018	59	0.39%	自建库
7	互联网论文-81366	57	0.38%	互联网
8	CIE 最前沿 面向FPGA的布局与布线技术研究综述-中国电子学会	52	0.34%	互联网
9	基于遗传算法和模拟退火算法的餐具配送企业运输路径优化 方苏俊 - 2023	45	0.30%	自建库
10	基于深度强化学习的FPGA布局算法优化研究 唐毅龙 - 2023	41	0.27%	博硕

相似文献汇总 (当前只展示10条数据,全部详情请查看片段对照报告)

相似文献来源：41

序号	相似文献	相似字符数	相似率	来源
1	面向FPGA的布局与布线技术研究综述 田春生;陈雷;王源;王硕;周婧;张瑶伟;庞永江;周冲;马筱婧;杜忠 - 电子学报 - 2022	962	6.34%	期刊

2	图划分方法及其在分布式网络环境下的应用 马永刚 - 2012	232	1.53%	博硕
3	流式处理的异步图处理框架 李金吉;张岩峰;巩树凤;于戈;高立新 - 软件学报 - 2018	178	1.17%	期刊
4	动态图划分算法研究综述 李贺;刘延娜;袁航;杨舒琪;韵晋鹏;乔少杰;黄健斌;崔江涛 - 软件学报 - 2023	151	1.00%	期刊
5	多FPGA系统结构感知的电路快速划分方法研究 李本正 - 2023	137	0.90%	博硕
6	科普中国-FPGA- · 科普中国网	112	0.74%	互联网
7	基于ZYNQ的2.5GSPS数据采集模块硬件设计 张波 - 2025	79	0.52%	博硕
8	单晶炉液位检测的曲面镜拟合方法研究 何恒盛 - 2016	77	0.51%	博硕
9	高速数字图像采集系统数据采集传输技术的研究 陈伟 - 2008	77	0.51%	博硕
10	国产替代核心力量，紫光国微凭什么？ fpga_网易订阅	69	0.45%	互联网

文字标注

■ 自写片段
 ■ 复写片段
 ■ 引用片段
 ■ 专业术语
 ■ 自引片段

福州大学

本科生毕业设计（论文）

题 目： 面向亿门级FPGA电路的划分算法研究

姓 名： 张驭龙

学 号： 032201115

学 院： 物理与信息工程学院

专 业： 信息与计算科学（数理综合班）

年 级： 2022级

校内指导教师： （签名）

校外指导教师： （签名）

2026年 5月 10日

福州大学本科生毕业设计（论文）诚信承诺书

学生姓名	张驭龙	年 级	2022	学 号	032201115
所在学院	物理与信息工程学院			所学专业	信息与计算科学
毕业设计（论文）题目		中文：面向亿门级FPGA电路的划分算法研究			
		外文：Research of Partitioning Algorithm for Billion-gate FPGA			
学生承诺 我承诺在毕业设计（论文）过程中遵守学校有关规定，恪守学术规范，未存在买卖、代写、作假等行为。在本人的毕业设计（论文）中未剽窃、抄袭他人的学术观点、思想和成果，未篡改实验数据。如有违规行为发生我愿承担一切责任，接受学校的处理。 学生（签名）： 年 月 日					
我承诺在指导学生本人认真的核查，该同剽窃、抄袭他人的学术		指导教师承诺			
		业设计（论文）过程中遵守学校有关规定，恪守学术规范，经过未存在买卖、代写、作假等行为，毕业设计（论文）中未发现有点、思想和成果的现象，未发现篡改实验数据。 指导教师（签名）： 年 月 日			

面向亿门级FPGA电路的划分算法研究

摘要

现场可编程逻辑门阵列（Field Programmable Gate Array, FPGA）是作为专用集成电路领域中的一种半定制电路而出现的，因其具有更高的集成度、更强的逻辑功能和更大的灵活性，目前已成为设计数字电路或系统的首选器件之一。正因如此，对与FPGA相关的研发、设计技术的需求与日俱增，其中便包括面向亿门级的多芯片FPGA电路的划分算法的研究。

面向亿门级的多芯片FPGA电路的划分算法的基础是超图划分技术。国内外在超图划分领域的研究成果已十分丰富，从而产生了许多可行且有效的多芯片FPGA电路的划分算法。未来随着科技的发展与人类的需要，FPGA电路将会更加集成，为顺应未来发展趋势，本课题便是在国内外研究成果的基础上实现将现行可用的划分算法更新迭代为可完成大规模划分任务的可行且有效的算法。

该算法总共分为三步：准备工作、初始划分、优化方案。其中准备工作为将所需数据数字化并生成相应写有超图信息的文件用于第二步操作；初始划分为使用Catalyurek U等人提供的闭源可运行的PaToH (Partitioning Tool for Hypergraph) 对数据进行划分得到初始划分结果；优化方案为对第二步生成的初始划分结果基于不同芯片内部资源利用率的平均差异百分比和各个芯片间的物理连线条数的限制，以贪婪放置 (Greedy Placement) 的思路进行优化，最后得出符合条件的划分结果。

通过将某数据划分到 Xilinx VU13P(4-dies)的实践任务，验证了该算法在万门级多芯片 FPGA 电路的划分任务方面的可行性和有效性，并基于该实践提出不足与对应解决思路，推理分析该算法在亿门级划分任务中的可行性。

关键词：FPGA，划分，C++，集成电路，组合优化

Research of Partitioning Algorithm for Billion-gate FPGA

Abstract

Field Programmable Gate Array(FPGA) is created as a semi-custom circuit in the field of application-specific integrated circuits, because of its higher integration, stronger logical functions and greater flexibility, it has been one of the preferred devices for designing digital circuits or systems. Since the reason mentioned above, there is an increasing demand for R&D (Research and Development) and design technologies related to FPGAs, including the research of partitioning algorithm for billion-gate FPGA.

The theoretical basic of the partitioning algorithm for billion-gate FPGA is the hypergraph partitioning. From home and abroad, there has been many achievements in the field of hypergraph partitioning, which has led to the development of many feasible and effective partitioning algorithms for multi-chip FPGA circuits. In the future, with the development of technology and human's needs, FPGA will be more integrated, to adapt to this trends, this research is going to update the current available partitioning algorithm into a feasible and effective algorithm capable of handling large-scale partitioning tasks.

This algorithm contains three steps: preparation, first-time-partitioning, optimization. Preparation is to digitalize the data we needed and make a file used in step two, containing the information of the hypergraph. First-time-partitioning is to get the first-time partitioning result by using the closed-source and executable PaToH (Partitioning Tool for Hypergraph) provided by Catalyurek U et al. Optimization is to optimize the first-time partitioning result considering the average percentage difference in internal resource utilization across different chips and the limitations on the number of physical inter-chip connections(SLL) using Greedy Placement approach.

By applying the algorithm to the partitioning task base on Xilinx VU13P(4-dies), its feasibility and effectiveness in partitioning tasks for million-gate multi-chip FPGA circuits were verified. Based on this practical application, shortcomings were identified along with corresponding solutions, and the feasibility of the algorithm for billion-gate partitioning tasks was analyzed through reasoning.

Keywords: FPGA; partitioning; C++; integrated circuit;
combinatorial optimization

目录

目录 1

第一章 绪论 3

1.1 引言 3

1.2 研究现状 3

1.3 FPGA电路的划分问题定义 6

1.4 课题目标 7

第二章 算法设计与研究 9

2.1. 算法流程设计 9

2.2. 数据结构 10

2.2.1 数据文件构成 10

2.2.2 数据结构设计 10

2.3. 初始划分 11

2.3.1 划分原理（超图划分理论） 11

2.3.2 PaToH 划分器的使用 12

2.4. 优化算法 13

2.4.1 约束条件（划分阈值） 13

2.4.2 贪婪放置（Greedy Placement） 15

2.4.3 基于不同芯片内部资源利用率平均差异的阈值的优化 15

2.4.4 基于各个芯片间物理连线的阈值的优化（以“链式”数据流设计为例） 18

第三章 算法实践检验 20

3.1. 数据数字化 21

3.2. 初始划分与约束相关参数 21

.....

3.3. 优化与结果 22

第四章 总结 24

4.1. 课题总结 24

4.2. 未来展望 24

附件 26

参考文献 27

附录1 30

致谢 50

第一章 绪论

1.1 引言

现场可编程逻辑门阵列（Field Programmable Gate Array，下称FPGA）是一种可反复编程的硬件芯片。它内含大量可配置逻辑单元和可编程连线，用户能通过特定的硬件描述语言定制专用的电路。相较于CPU的指令的串行执行，FPGA凭借硬件的并行的特性实现低延迟、高吞吐，且功耗可控的性能，这种“硬件可重构”的能力使得其被广泛地应用于诸如5G通信、图像处理、金融加速、高速数据采集等等多个场景中。FPGA电路不仅解决了定制电路的不足，还克服了原有可编程器件门电路数有限的缺点，因其具有更高的集成度、更强的逻辑功能和更大的灵活性，目前已成为设计数字电路或系统的首选器件之一。

近年来，FPGA产业在AI与边缘计算浪潮推动下实现了快速增长^[1]。全球市场规模已从2025年的约125亿美元攀升至2026年的143亿美元，年复合增长率高达14.3%。市场高速扩张的背后，是各行业“硬件可重构”的能力的需求正从传统数字系统设计全面转向AI/ML硬件加速、自动驾驶ADAS以及5G/6G通信基础设施的设计。

在技术层面，FPGA电路正在经历从单一可编程逻辑器件向异构计算平台的转型。厂商纷纷推出集成AI Tensor Block、DSP及嵌入式处理器内核的自适应SoC（如Chiplet级异构集成），在保持灵活性的同时大幅提升算力效率^[20]^[21]。硬件可信根、后量子密码等先进的安全特性也被深度集成，从而满足一些高安全场景的应用需求。

在应用层面，FPGA已从传统的原型验证拓展转变为数据中心智能网卡与AI推理加速的关键器件^[22]^[23]。在AI大模型训练中，FPGA可用于梯度压缩与通信加速，缓解GPU集群的带宽瓶颈；在边缘侧，其确定性的低延迟实时处理能力在机器人与医学成像等领域得到广泛验证。在5G/6G通信领域^[19]，FPGA更是基站前传接口与核心网设备的理想选择。

因此，对与FPGA相关的研发、设计技术的需求与日俱增。

1.2 研究现状

与FPGA相关的研发、设计技术包括FPGA的布局技术和FPGA的布线技术等等。

其中，有关FPGA的布局技术，目前已存在许多种不同的方法策略，主要可以分为三种：基于划分的布局技术、基于启发式的布局技术以及基于解析式的布局技术^[1]。

基于划分的布局技术是指根据已知的布局约束关系，在最初始的布局方案的基础上，通过递归迭代来对布局方

案进行划分，直至得到的布局方案小于事先设定的阈值为止^{[1][5]}的FPGA布局技术。以Kim等人研究设计的SimPL布局方法^[31]为代表，该方法以线长作为约束条件，分别计算线长的下界与上界，然后通过一种基于自上向下的几何分割和非线性缩放的前向合法化方法不断进行迭代，最终使逻辑单元收敛到合法的位置。基于划分的布局技术的优点在于其将复杂的电路设计分割为较简单的电路设计，从而能够快速对局部电路进行处理，并且其收敛速度快。

基于启发式的布局技术是指以一些自然现象为设计原理，对FPGA逻辑单元的布局过程进行数学建模^[1]的FPGA布局技术。在这方面的典型代表有SADEGHI A等人所研究的遗传算法^[24]、HERATH K等人所研究的蚁群算法^[25]以及模拟退火算法

^{[26][27][28]}，其中又以模拟退火算法为基于启发式的布局技术的研究的热门方向。1953年，Metropolis 等人^[27]最早提出了模拟退火算法理论，它是一种基于 Monte-Carlo 求解方式的随机搜索算法，其原理是物理中固体物质的退火过程与一般组合优化问题之间的具有一定的相似性，故可通过模拟固体物质的退火过程来解决部分组合优化问题。到了1988年，Carl 等人^[28]首次将模拟退火算法的思想引入到超大规模集成电路（Very Large Scale Integration, VLSI）布局领域。随后，越来越多的研究学者开始关注模拟退火算法在 VLSI、FPGA 等布局领域的应用。目前学界较为流行的FPGA 通用布局布线（Versatile Placement and Routing, VPR）工具的布局器的基础框架正是模拟退火算法。基于启发式的布局技术的优点在于其相对于最优算法，在时间允许的情况下，能够获得布局的最优解。

基于解析式的布局技术是指将FPGA布局问题建模为一个连续优化问题，通过梯度下降的方法得到布局问题的最优解^[1]的FPGA布局技术。以目标函数为分类标准，该布局方法共分为两种：GOPALAKRISHNAN P等人所研究使用的二次方布局技术^[29]、LIN T H等人所研究使用的非线性布局技术^[30]。二次方布局技术通常以线长为优化目标完成对目标函数的建模，随后通过Krylov子空间方法将上述优化问题转化为对称与正定（Symmetric and Positive-Definite, SPD）线性系统的求解问题，最终得到优化问题的解；非线性布局技术则使用不可微分的非线性函数来表示非重叠的约束条件，并将其与线长一起在统一的目标函数中进行求解。基于解析式的布局技术的优点在于其完全依赖于数学方法，其最终的布局质量能够通过数学分析来进行证明，具有较高的布局质量，并且与基于划分的布局方法一样收敛速度较快。

多芯片FPGA电路的划分技术本质上是对超图的划分技术，因此图划分与超图划分技术的进步促进了多芯片FPGA电路的划分技术的不断迭代。在图划分和超图划分领域中已有不少成果^[7]，如：

1) 2012年马永刚改进的图划分模型^[10]。针对图划分应用于分布式并行计算只能表达对称和双向依赖关系的问题，提出了基0-1于规划方法的并行计算图划分模型；针对标准图划分模型应用于并行计算不能准确表达通信量，也不能考虑通信延迟和通信量的分布情况对并行性能的影响，提出了改进的图划分模型；针对分布式网络控制中的工业网络优化设计问题，提出了基于非线性0-1规划理论对其进行建模；为了在分布式交通控制中有效的聚合查询处理，路网数据应该有效的组织和存储，针对这一问题提出了基于聚类图模型的改进方法；

2) 2018年李金吉等人的流式处理的异步图处理框架^[12]。李金吉等人结合了累加迭代计算和单机并行处理技术，提出流式处理的异步计算模型 ASP。它实现了对磁盘的完全顺序访问，允许流式的顺序载入结构数据的同时进行异步更新计算。基于 ASP模型，又提出了一种流式处理的异步图处理框架 S-Maiter，实现了高效率的基于外存的单机大图处理，通过I/O 线程优化、内存资源监控、shard 级优先级调度等优化技术，提高了系统处理大图数据的性能。；

3) 2023年李贺等人对动态图划分算法的研究整理^[13]。其中包括流式图划分算法、增量式图划分算法和图重划分算法。流式图划分算法包括基于 Hash 的划分算法、基于邻居分布的划分算法以及基于流的优化划分算法，增量式图划分算法包括单元素增量式划分算法和批量增量式划分算法，图重划分算法包括针对图结构动态的重划分算法和针对图计算动态的重划分算法；

4) 2025年路文军等人的宽度优先超图划分技术^[8]。该算法增加一个增益函数使宽度优先算法中的顶点除按照与原始顶点相关性强弱进行优先级排列外，对相关性强弱相同的顶点根据增益值的大小进行排列生成初始划分结果，然后借鉴贪心划分算法的思路，在保证可行性的基础上将部分顶点按照宽度优先的优先级排列划分到两块中，对于剩余的自由顶点根据增益值进行划分；

5) ÇATALYÜREKÜ等人^[7]、尚俊霖等人^[11]、Fischer E等人^[14]以及BuluçA等人^[15]对2025年以前的（超）图划分技术的综述。详细（超）图划分理论可参考2.3.1。

在前述超图划分理论基础的便诞生了多芯片FPGA电路的划分算法^{[1][3][4]}，并且出现了许多基于上述理论的实际案例，如：

1) 2005年KHATKHATE A等人的基于递归二分法的布局技术^[5]。他们将传统的递归二分标准单元格放置工具扩展到直接考虑混合块设计；

2) 2023年李本正的多FPGA系统结构感知的电路快速划分方法^[9]。他针对 RTL电路划分的问题，提出了一种将RTL电路映射为超图的框架；提出了一种面向MFS结构的、高性能的、可以避免 FPGA 间信号中转的电路划分方法；在避免 FPGA 间信号中转的电路划分方法的基础上，提出了一种新型的 MFS 感知划分框架；针对学术与工业界急需一个高效的面向 MFS 大电路划分器的问题，将提出的划分算法部署成为了可以实际应用的划分器工具；

3) 2026年黄宏赞等人的多维感知的多芯片FPGA划分方法^[2]。他们针对跨芯片时序约束和物理拓扑约束的挑战，基于多级范式提出了一种多维约束感知的多芯片FPGA网表划分方法，突破通用工具仅平衡分割的局限。

1.3 FPGA电路的划分问题定义

本课题为面向亿门级FPGA电路的划分算法研究，即本课题所研究的算法要实现将亿门级的节点数合理地分布在多芯片FPGA电路（如Xilinx VU13P(4-dies)）的芯片上。简单来看，假设节点数为 n ，目标多芯片FPGA电路的芯片数为 k ，则该算法便是用来实现将 n 个节点根据特定的约束条件划分到 k 个芯片中。

正如1.2所述，多芯片FPGA电路的划分本质上是超图划分。我们假设一个超图 $H=(V,N)$ ，那么我们可以把 n 个节点看作超图中的、 n 个节点的相互关系所构成的超边看作超图中的，芯片数 k 看作我们的划分目标路数 K ，则该算法所实现的功能便是对上述超图 H 进行 K 路划分。

超图划分相关概念与定义如下：

定义1 无向超图 $H(V, N, c, \omega)$ 由一个顶点集 V 和一个超边集 N 组成，并且顶点的权重为 $c: V \rightarrow R \geq 0$ ，超边的权重为 $\omega: V \rightarrow R \geq 0$ 。取 $v \in V, n \in N$ ，若 $v \in n$ ，则表示顶点 v 与超边 n 相邻。若 $n \in N: v \in n \wedge u \in n$ ，则表示顶点 v 和 u 相邻。记 $\Gamma(v) = \{n \in N : v \in n\}$ 表示与顶点 v 相邻的所有超边 n 的集合。记 $\Delta(v) = |\Gamma(v)|$ 表示顶点 v 的度。若 $\Delta(v) = 0$ ，则称顶点 v 为孤立点。记 $\Gamma(n) = \{v \in V : v \in n\}$ 表示与超边 n 相邻的所有顶点 v 的集合。记 $|\Gamma(n)|$ 表示超边 n 的大小。

若 $|V_i| = 1$ ，则称超边 n 为单节点超边。若 $V_i = V_j$ ，则称超边 n 与 m 是并行的。

定义2 超图 H 的 K 路划分 $P = \{V_1, V_2, \dots, V_K\}$ 的顶点集 V 划分成 K 个部分，且满足约束条件：

$$1) \bigcup_{i=1}^K V_i = V \wedge \forall V' \in P: V' = \emptyset;$$

$$2) \forall i, j \in [1, 2, \dots, K]: i = j \Rightarrow V_i \wedge V_j = \emptyset;$$

$$3) \forall V' \in P: c(V') \leq L_{\max}.$$

其中 $L_{\max} = \left\lceil \frac{c(V_i)}{K} \right\rceil (1 + \varepsilon) + \max_{v \in V} c(v)$ 的。

定义3 一个超边 n 在块中的顶点数记为 $\lambda(n) = |\{v \in V \mid v \in n\}|$ 。若 $\lambda(n) > 0$ 则说明超边 n 与块有关。若 $\lambda(n) = 0$ 则说明超边 n 与块无关。超边 n 的连通集记为 $\Lambda(n) = \{i \mid \lambda(n) > 0\}$ 。记 $|\Lambda(n)|$ 表示超边 n 的连通集的大小。若 $|\Lambda(n)| > 1$ ，则称超边 n 为切边。若 $|\Lambda(n)| = 1$ ，则称超边 n 为内部边。

定义4 超图 H 的划分 P 的切边度量的计算公式如下，

$$\kappa(H, P) = \sum_{n \in N \wedge \lambda(n) > 1} \omega(n)$$

超图划分的目标便是最小化切边度量 $\kappa(H, P)$ 。

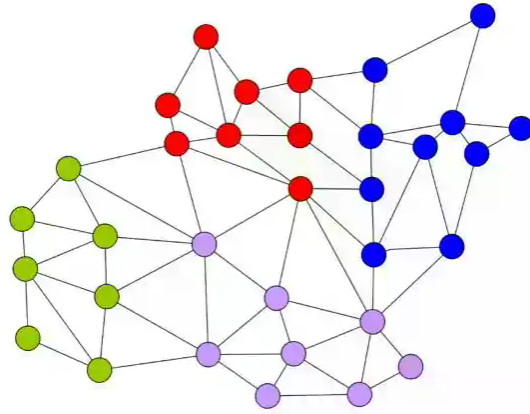


图 1 图划分示意图

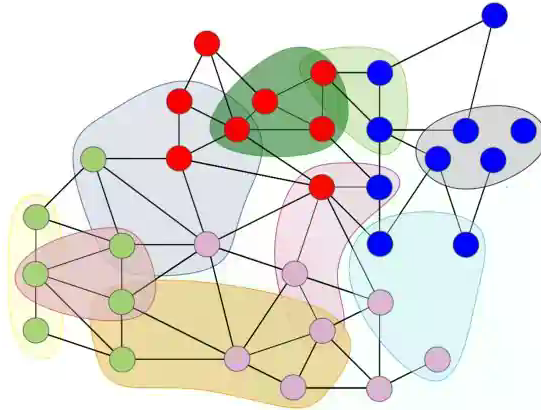


图 2 超图划分示意图

1.4 课题目标

本课题旨在在国内外前辈的研究成果的基础上，探索并设计出一套能实现将亿门级数据划分到多芯片FPGA电路的划分算法，并利用C++代码实现该算法，通过简单的实践任务来检验该算法的可行性和有效性。

全文共分为四个部分，分别为，

- 1) 第一章为绪论。主要内容为本课题的研究背景介绍、国内外与本课题相关的研究成果的收集以及本课题的研究意义与计划。
- 2) 第二章为算法设计与研究。主要内容为针对本课题的多芯片FPGA电路的划分算法的分步设计及其相应理论研究与逻辑推导。
- 3) 第三章为实践检验。主要内容为使用第二章设计出的算法完成将某组数据划分到Xilinx VU13P(4-dies)的四个芯片上，并分析结果，判断任务是否完成。
- 4) 第四章为总结。主要内容为基于第三章的实践检验结果判断算法的可行性与有效性以及本课题存在的不足之处与相应解决方向，最后对本课题的未来发展进行展望。

第二章 算法设计与研究

2.1. 算法流程设计

该算法所实现的功能为：输入一份数据文件，输出对应的划分结构。故可设计出具体的实现流程如图3所示。

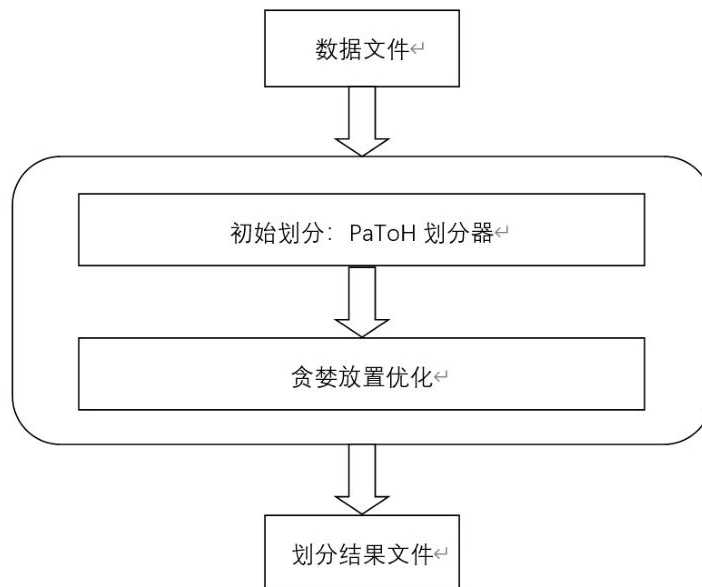


图 3

图1中，该算法共分三步走：

- 1) 提取信息。数据文件经过数字化处理后获取所有信息（用于存放数据文件相关信息的数据结构设计详见2.2），并如1.3所述构建超图文件；
- 2) 初始划分。使用PaToH划分器（详见2.3）将超图文件进行初始划分得到初始划分结果；

3) 优化。根据所需限制条件（详见2.4），结合贪婪放置的优化思想得到最终项目所需的划分结果。

2.2. 数据结构

2.2.1 数据文件构成

原始数据文件包含10个子文件，其中本课题所用到的文件包括Object File Library（.lib）、NETS（.nets）和NODES（.nodes）文件，详见附件1。

Object File Library文件内容为该数据所用资源（cell）以及各个cell的引脚（pin）类型与对应的输入输出类型。以图4的(a)为例。

NETS文件内容为该数据的网络（net）信息，包含net的名称、net中的节点（node）以及node对应的引脚类型。以图4的(b)为例。

NODES文件内容为该数据的节点（node）信息，包含node的名称以及该节点所消耗的资源（cell）。以图4的(c)为例。

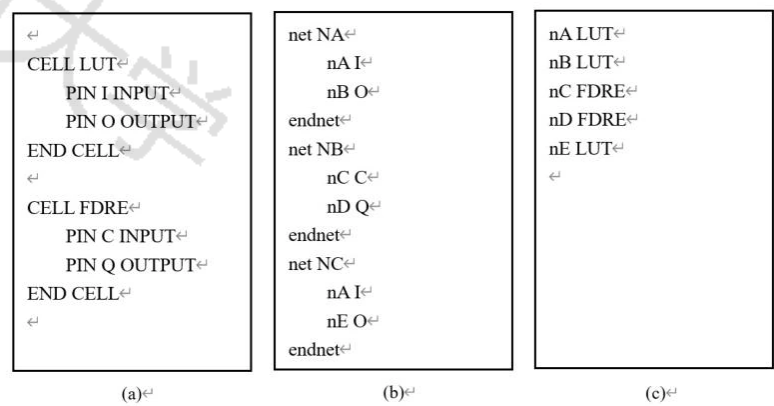


图 4(a)Object File Library文件示例，如图中有两种cell，对应的输入引脚I和 C，输出引脚C和Q；(b)NETS文件示例，如图中有三个 net，其中NA 有输入节点nA和输出节点 nB、NB有输入节点nC and 输出节点nD、NC有输入节点nA和输出节点nE；(c)NODES文件示例，如图中有五个节点以及其对应的cell。

2.2.2 数据结构设计

在前述数据文件构成基础上，设计数据结构如下表1。

类 (class)	变量	说明
cell	string name	cell的名称
die	int capacity int list[5000]	die中的node的个数die中的node的索引
net	string name	net的名称
	string name	node的名称
node	int cell	node所消耗的cell的索引
	int die	node所在的die的索引

表 1

如表1所示，设计了四个类，分别为cell，die，net以及node。

类cell包含的变量有name（cell的名称）。

类die包含的变量有capacity（die中node的数量），list（die中有哪些node）以及加入die函数和从die中删除的函数。

类net包含的变量有name（net的名称）。

类node包含的变量有name（node的名称），cell（node所消耗的cell）以及die（node所在的die）。

具体C++源代码详见附件2中的cpp/cell.h，cpp/dies.h，cpp/net.h，cpp/nodes.h文件。

2.3. 初始划分

正如1.2与1.3所述，多芯片FPGA电路的划分问题本质上是超图划分问题。我们假设一个超图 $H=(V,N)$ ，那么数据文件中的node便是超图中的、net便是超图中的。如此一来，我们初始划分的目的，便是将数据文件中的node根据net的结构进行一次超图划分操作从而将数据文件中的node初步划分到die中，目标die有K个，我们便将数据文件中的node划分成K个部分。

在对不同划分工具^{[6][17]}之间进行对比后，我选择了由Catalyurek U教授和Aykanat C教授主导开发的PaToH (Partitioning Tool for Hypergraph)^[17]来对数据进行初始划分。

2.3.1 划分原理（超图划分理论）

超图划分是将超图 $H=(V,N)$ 划分为K个部分，即所谓超图H的K路划分（k-ways partitioning），其目标是找到一个平衡划分方案使得割（cut）的代价（cost）越小越好。平衡划分的约束条件为平衡约束，即每个部分的顶点总权重不超过平均权重的 $1+\epsilon$ 倍。

PaToH划分器采用两种方式计算代价：一种是切边度量，即所有被切割超边的代价之和；一种是连通性度量，即每条被切割超边的代价乘以其连通部分数减1。

如图5所示，超图划分的三个阶段如下，

1) 粗化。将原始超图逐级压缩成越来越小的超图。利用聚类算法将多个顶点合并为一个顶点，并收缩内部超边，逐步减少问题规模，直到顶点数降至阈值，得到最粗的小超图。PaToH 实现了多种聚类算法^[16]，包括基于匹配的聚类和凝聚式聚类。

2) 初始划分。在最粗的小超图上，运行贪心超图生长算法（Greedy Hypergraph Growth Algorithm）生成初始二分划分结果。该算法从一个随机顶点开始，利用类似FM的增益优先队列逐步“生长”出一个满足平衡约束的簇。由于粗图很小，会多次随机运行，选择最优结果进入下一阶段。

3) 细化。将粗超图的二分划分方案逐级映射回更精细的超图。在每一级映射后，都会运行基于KL/FM（Kernighan-Lin/Fiduccia-Mattheyses）的局部优化算法（如

BFM，即边界FM）来优化当前的划分方案。这个过程可在多个层级上形成一个“V-循环”进行反复优化，直到恢复原始超图，得到最终的K路划分结果。

综上所述，PaToH划分器的划分策略为递归二分法，对于 $K>2$ 的情况，PaToH划分器采取多次二分超图来实现多路划分。



2.4.1 约束条件（划分阈值）

受本人在课题研究的现实约束，本课题的约束条件（划分阈值）有两个：不同芯片内部资源利用率的平均差异百分比与各个芯片间的物理连线条数。

不同芯片内部资源利用率的平均差异：由于各个芯片内部的资源是有限、相同且等量的，记该指标为 a ，则 a 的计算公式如下，

$$a = \frac{C_{max} - C_{min}}{C_{min}}$$

其中， C_{max} 和 C_{min} 分别为所有芯片中划分到最多node的die中的node数和划分到最少node的die中的node数。

各个芯片间物理连线：参考文献^[5]^[9]，该指标的计算方法因多芯片FPGA电路布局设计者对数据流的不同设计而不同。通常有以下数据流设计，

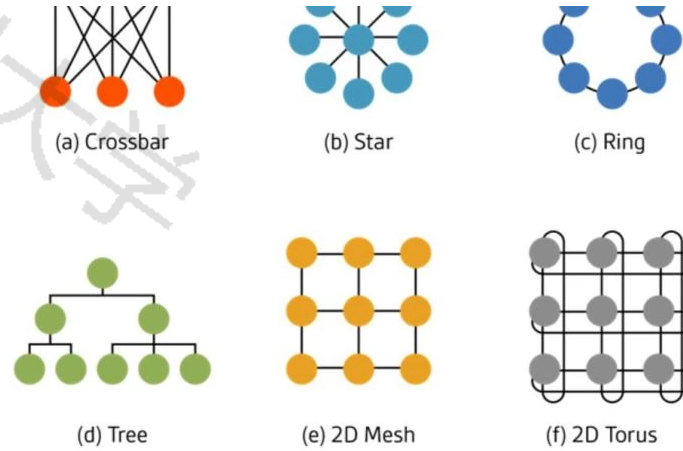


图 7 各种数据流设计

1) “星形”（Star）数据流^[18]：如图7中的(b)所示，由一个node向其它node传递数据，故数据统一从一个die传递到另一个die，则该设计下的物理连线条数 l 的计算方法如下，

$$l = \sum_i node_{net_i} - 1$$

其中， $node_{net_i}$ 为第 i 个net中的node的个数。

2) “链式”（Chain）数据流^[18]：类似图7中的(c)所示，net中各个node所在的die以链条的形式“串起来”，数据在链条上一个die一个die的传递，则该设计下的物理连线条数 l 的计算方法如下，

$$l = \sum_i Die_{net_i} - die_{net_i}$$

其中， Die_{net_i} 和 die_{net_i} 分别为与net相交的所有die（net中部分node被划分到的die）中索引的最大值和最小值。

3) “网格”（Mesh）数据流^[18]：如图7中的(e)所示，数据在die间的传递如同网格一般，则该设计下的物理连线条数较难计算。

本课题基于以上两个约束条件，以贪婪放置（Greedy Placement）为基础对优化算法进行设计。

2.4.2 贪婪放置（Greedy Placement）

假设我们得到的初始划分结果的不同芯片内部资源利用率的平均差异大于阈值（由于PaToH划分器^[17]的划分效果，我们的初始划分方案为平衡划分，故不同芯片内部资源的平均差异大部分情况下不会超过阈值），以贪婪放置为设计基础，我们可以把划分到更多node的die拿出部分移到划分到更少node的die中。具体设计详见2.4.3。

假设我们得到的初始划分结果的各个芯片间物理连线大于阈值，以贪婪放置为设计基础，我们可以找到所需物理连线最多的net，将与其相交的所有die中最远的die中的node移到近一点的die上。具体设计详见2.4.4。

不难发现，在上述假设中，只要我们在基于不同芯片内部资源利用率的平均差异进行优化时合理地将部分node换到新的die中，得到的划分方案的各个芯片间的物理连线条数可以不会发生改变，但我们在基于各个芯片间的物理连线条数进行优化时，我们无法保证得到的划分方案的不同芯片内部资源利用率的平均差异保持不变。

因此，基于上述推理研究所得，在优化优先级上我们应先基于各个芯片间的物理连线进行优化，再基于不同芯片内部资源利用率的平均差异进行优化（即实际操作时，先进行2.4.4中的操作，再进行2.4.3中的操作）。

2.4.3 基于不同芯片内部资源利用率平均差异的阈值的优化

顺着2.4.2中基于不同芯片内部资源利用率的平均差异的优化方法的思路，我们可以设计出如下优化算法，

输入：NETS文件，一个node类的数组对象node_list，一个die类的数组对象die_list。

输出：新的node_list，新的die_list。

//a) 设置参数

d_sorted[], net_in_die[], sum_net_in_die= 0, net_C_M =0, i_net_C_M = -1;

初始化参数;

以die_list.capacity为基准从大到小排序，并将对应die的索引按顺序存入d_sorted; //b) 寻找拥有最多node的net
一行一行读取NETS文件:

空行跳过;

每行第一个字符串为“net”：跳过;

每行第一个字符串为“endnet”：

若 $net_in_die[d_sorted[0]]=1$;

若此时的node的索引指针大于net_C_M:

则net_C_M为此时的node的索引指针的值且i_net_C_M为此时net的索引;

重置net_in_die, sum_net_in_die和node的索引指针后跳过;

记本行node在node_list的索引为r;

net_in_die[node_list[r].die]=1;

node的索引指针自增;

//c) 变更node所在的die

重置net_in_die;

mode= -1;

一行一行读取NETS文件:

空行跳过;

每行第一个字符串为“net”: 跳过;

每行第一个字符串为“endnet”:

若此时net的索引为i_net_C_M:

sum_net_in_die为net_in_die中所有数值的和;

若sum_net_in_die不为1: 则mode=0;

否则: mode= 1;

退出文件;

否则: 跳过;

若此时net的索引不是i_net_C_M: 则跳过;

若此时net的索引是i_net_C_M:

记本行node在node_list的索引为r;

net_in_die[node_list[r].die]= 1;

一行一行读取NETS文件:

空行跳过;

每行第一个字符串为“net”: 跳过;

每行第一个字符串为“endnet”:

若此时net的索引为i_net_C_M: 退出文件;

否则: 跳过;

若此时net的索引不是i_net_C_M: 则跳过;

若此时net的索引是i_net_C_M:

若mode= 1:;

将本行的node转移到第d_sorted[MAX]个die;

若mode= 0:

若本行的node所在die为d_sorted[0]: 则将这个node优先转移到第d_sorted的索引越大的数值个die中, 且该die所对应的net_in_die为1;

//d) 更新die中的相关信息

更新die_list;

步骤a)中, 设置了d_sorted, net_in_die, sum_net_in_die, net_C_M以及

i_net_C_M四个参数(数组), 分别用来存放die所有的node数的从大到小排序后的索引顺序, net与die相交与否(相交为1, 否则为0), net_in_die中所有数值的和(表示net与多少个die相交, 用于确定如何对node进行移动操作, 详见步骤c)), 所有net中拥有最多node的net的node所有数以及其对应的索引。

步骤b)中，读取NETS文件来获得各个net的node信息，再根据node_list的数据信息找到所有net中拥有最多node的net的node所有数（net_C_M）以及其对应的net的索引（i_net_C_M）。

步骤c)中，第一遍读取NETS文件来确定与所有net中拥有最多node的net相交的die的总数sum_net_in_die，若该值为1，则使用方案2进行node转移（即将node从旧的die移动到新的die中），否则使用方案1。在第二遍读取NETS文件时，选择不同方案进行操作：

方案1，将在第d_sorted[0]个die上的node优先移动到对应d_sorted索引越大且与本net相交的die中；（这样操作可以确保各个芯片之间的物理连线条数保持不变）

方案2，将net中所有node移动到d_sorted索引最大的数值所对应的die中。（有我们所设计出的优化算法可推理出该net中所有的node均在第d_sorted[0]个die中，本方案相当于把位于拥有最多node的die中的net整个移动到拥有最少node的die中）

2.4.4 基于各个芯片间物理连线的阈值的优化（以“链式”数据流设计为例）

顺着2.4.2中基于各个芯片间的物理连线的优化方法的思路，我们可以设计出如下优化算法，

输入：NETS文件，一个node类的数组对象node_list，一个die类的数组对象die_list。

输出：新的node_list，新的die_list。

//a) 设置参数

d_M=0, d_m=3, net_SLL_M=0, i_net_SLL_M = -1;

//b) 寻找所需物理连线最多的net

一行一行读取NETS文件：

空行跳过；

每行第一个字符串为“net”：跳过；

每行第一个字符串为“endnet”：

若d_M-d_m>net_SLL_M：

则net_SLL_M = d_M-d_m且i_net_SLL_M为此时net的索引；

重置d_M和d_m后跳过；

若本行的node所在的die更远：则更新d_M；

若本行的node所在的die更近：则更新d_m；

//c) 变更node所在die

重置d_M和d_m；

一行一行读取NETS文件：

空行跳过；

每行第一个字符串为“net”：跳过；

每行第一个字符串为“endnet”：跳过；
此时net的索引不是i_net_SLL_M：跳过；
此时net的索引是i_net_SLL_M：
若本行的node所在的die更远：则更新d_M；
若本行的node所在的die更近：则更新d_m；
一行一行读取NETS文件：
空行跳过；
每行第一个字符串为“net”：跳过；

d_M=0, d_m=3, net_SLL_M=0, i_net_SLL_M=-1; 若 d_M-d_m > net_SLL_M:

每行第一个字符串为“endnet”：跳过；

此时net的索引不是i_net_SLL_M：跳过；

此时net的索引是i_net_SLL_M：

若本行的node所在die为d_M：则把这个node转移到第d_M- 1个die
中；

（或者，若本行的node所在die为d_m：则把这个node转移到第d_m+
1个die中；）

//d) 更新die中的相关信息

更新die_list;

步骤a)中，设置了d_M, d_m, net_SLL_M以及i_net_SLL_M四个参数，分别用来存放net中的node所在的die的索引的最大值，net中的node所在的die的索引的最小值，所有net中所需物理连线最多的net的所需物理连线条数以及其对应的net的索引。

步骤b)中，读取NETS文件来获得各个net的node信息，再根据node_list的数据信息找到所有net中所需物理连线最多的net的所需物理连线条数（net_SLL_M）以及其对应的net的索引（i_net_SLL_M）。

步骤c)中，第一遍读取NETS文件来确定所有net中所需物理连线最多的net中的node所在的die的索引的最大值

（d_M）和最小值（d_m），有助于在第二次读取NETS文件时对该net中的部分node进行转移操作（即将node从旧的die移动到新的die中），要么把最远的node移近（即node_list[].die从d_M变为d_M-1），要么把最近的node移远（即node_list[].die从d_m变为d_m +1）。

第三章 算法实践检验

以附件3中的clk_design1为例进行实践检验。

实现本实践的划分算法的C++源代码详见附件2。

本实践是将clk_design1中的node划分到Xilinx VU13P(4-dies)中。该多芯片FPGA电路有四个芯片，并且是 1×4 排列（如图8所示），因此我选择使用“链式”数据流设计进行物理布局，这意味着在计算物理连线时，若

节点甲和节点乙相连且甲在第一个芯片而乙在第四个芯片，则需要3条物理连线才能将两个节点进行相连，依次类推。

本实践约束条件（阈值）为不同芯片内部资源利用率的平均差异 $\leq \pm 20\%$ ，且各个die之间的物理连线数（采用链式数据流策略）要求 ≤ 650 条。

具体操作流程详见图9。

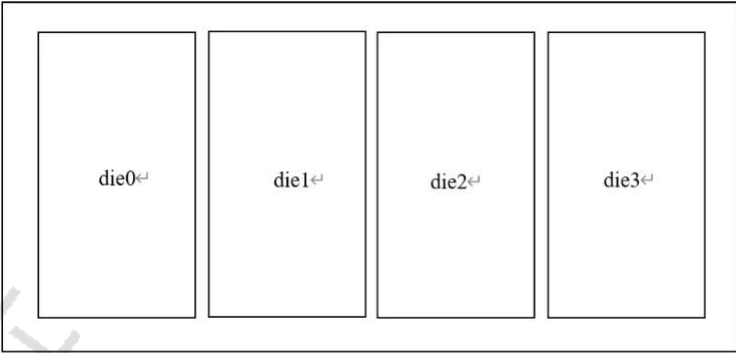


图 8

```
D:\毕设\毕设附件\2.算法cpp代码\cpp_code\data\clk_design1\design
Please select the mode:
0 for preparation, 1 for partitioning, 2 for placement, 3 for checking, 4 for optimization, others for exit.
0
Please select the mode:
0 for preparation, 1 for partitioning, 2 for placement, 3 for checking, 4 for optimization, others for exit.
2
Please select the mode:
0 for preparation, 1 for partitioning, 2 for placement, 3 for checking, 4 for optimization, others for exit.
3
The average cell using is: 0.00364964
The SLL using is: 660
Please select the mode:
0 for preparation, 1 for partitioning, 2 for placement, 3 for checking, 4 for optimization, others for exit.
4
Please input the requested average cell using and SLL:
0.2 650
Please select the mode:
0 for preparation, 1 for partitioning, 2 for placement, 3 for checking, 4 for optimization, others for exit.
3
The average cell using is: 0.0405126
The SLL using is: 784
Please select the mode:
0 for preparation, 1 for partitioning, 2 for placement, 3 for checking, 4 for optimization, others for exit.
5

D:\毕设\毕设附件\2.算法cpp代码\cpp_code\x64\Debug\毕设.exe (进程 67456)已退出。代码为 0 (0x0)。
要在调试停止时自动关闭控制台，请启用“工具”->“选项”->“调试”->“调试停止时自动关闭控制台”。
按任意键关闭此窗口。 . . ]
```

图 9 代码运行具体过程，C++源代码详见附件2，其中，操作0为前期准备、操作1是初始划分、操作2是完善数据、操作3为计算指标、操作4为优化，操作5为推出程序。

注：由于 PaToH 划分器为闭源项目，故本次实践检验前已完成初始划分操作，源代码文件仅拥有PaToH划分器的资源压缩包，文件名为.../2. 算法cpp代码/cpp_code/patoh-Linux-x86_64.tar.gz，文件来源为<https://cc.gatech.edu/~umit/software.html>。

3.1. 数据数字化

对应图9中从开始到输入0后再次弹出提示的过程。

输入文件名称（不加后缀），即输入 /clk_design1/design 。

成功输出对应HGR文件，详见附件3中的clk_design1/design.hgr文件。

经确认检验，所生产的HGR文件符合初始划分的使用标准，且可知clk_design1数据中有9882个node、9892个net以及13种cell。

3.2. 初始划分与约束相关参数

因某些因素未与图9任何操作对应，详见图4的注解。

输入文件名称（不加后缀）与划分路数，即输入 /clk_design1/design, 4。

成功输出对应文件，详见附件3中的clk_design1/design.hgr.part.4文件。

经检验，上述文件即为一种平衡划分方案，其中各个die中划分到的node分别有2475个、2466个、2471个、2470个。

各个die的资源利用情况及资源利用率如表2和表3所示。

计算同芯片内部资源利用率的平均差异百分比以及各个die之间的物理连线条数（详见图10）可知，初始划分后，不同芯片内部资源利用率的平均差异约为0.365%，

各个die之间的物理连线有660条。

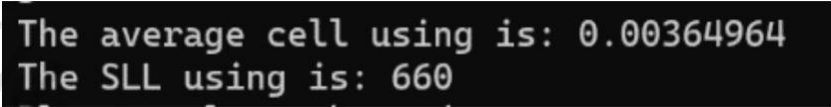


图 10 初始划分后的芯片内部资源利用率的平均差异与各个芯片间的物理连线条数

die 编号	LUT	FF	CARRY8	DSP48E2	RAMB36E2	IO
1	1245	1210	0	1	0	19
2	1240	1208	0	0	1	17
3	1229	1218	0	1	1	22
4	1209	1210	0	0	0	51
die可用资源	432000	864030	54000	3072	672	208

表 2 各个die的资源利用情况

die 编号	LUT (%)	FF (%)	CARRY8 (%)	DSP48E2 (%)	RAMB36E2 (%)	IO (%)
1	0.288	0.140	0	0.033	0	9.135
2	0.287	0.140	0	0	0.149	8.173
3	0.285	0.140	0	0.033	0.149	10.58
4	0.280	0.140	0	0	0	24.52

表 3 各个die的资源利用率

3.3. 优化与结果

对应图9中输入4后所有的操作。

输入阈值，即输入0.2，650。

成功输出对应文件，详见附件3中的clk_design1/design.hgr.part.4.optimized文件。

进行计算操作后，发现不同芯片内部资源利用率的平均差异小于阈值，为4.05%，但各个die之间的物理连线条数仍然超过阈值，为784条（详见图11）。

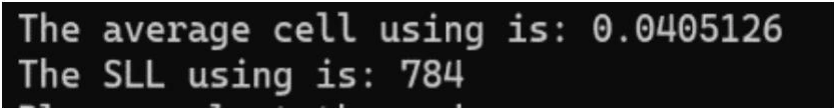


图 11 优化后的芯片内部资源利用率的平均差异与各个芯片间的物理连线条数

die 编号	LUT	FF	CARRY8	DSP48E2	RAMB36E2	IO
1	1245	1210	0	1	0	19
2	1240	1259	0	0	1	17
3	1229	1218	0	1	1	22
4	1209	1159	0	0	0	51
die可用资源	432000	864030	54000	3072	672	208

表 4 优化后各个die的资源利用情况。与表2 相比，优化后die2 和die4对资源FF的利用情况发生改变，die2对资源FF 的利用量从1208增加至1259，die4对资源FF 的利用量从1210下降至1159。

die 编号	LUT (%)	FF (%)	CARRY8 (%)	DSP48E2 (%)	RAMB36E2 (%)	IO (%)
1	0.288	0.140	0	0.033	0	9.135
2	0.287	0.146	0	0	0.149	8.173
3	0.285	0.140	0	0.033	0.149	10.58
4	0.280	0.134	0	0	0	24.52

表 5 优化后各个 die 的资源利用率。同理，与表 3 相比，die2 对资源 FF 的利用率从 0.140%增加至 0.146%，die4对资源FF的利用率从0.140%下降至0.134%。

对原始数据文件进行分析研究发现：同一个node可以属于多个net。这个发现可以推出：在优化过程中，对某一个net的node进行转移操作时，虽然该net的物理连线变少了，但其它net的物理连线可能增加了，若增加到更多，就会导致优化后的划分方案的各个die之间的物理连线反而增多的情况。

因此，我选择clk_design1/design.hgr.part.4文件作为最优的划分方案。

第四章 总结

4.1. 课题总结

通过本课题的研究，我了解了多芯片FPGA电路的划分的意义与原理，还包括作为多芯片FPGA电路的划分算法的理论支撑的（超）图划分，并在国内外前辈们的研究成果的基础上，完成了将原始数据中的clk_design1相关数据划分到Xlinx VU13P(4-dies)的四个芯片中的任务，将划分结果的不同芯片内部资源利用率的平均差异控制在0.365%左右，各个die之间的物理连线控制在660条，一定程度上证明了这套多芯片FPGA电路的划分算法在万门级情况下的可

行性和有效性。不过，本课题还存在不少可以改进的地方，例如，

- 1) 实践检验是，初始划分结果较好，致使算法中的优化部分的有效性的可信度不高。
- 2) 算法设计时是分步设计，并未将整个划分流程合并。
- 3) 受硬件影响，该算法还未在node数超过10000的数据，即引脚数不超过万门级的多芯片FPGA电路的划分任务中实践检验。

相应的研究思路为，

- 1) 两种思路：一种选择比PaToH划分器较弱的划分器进行初始划分操作；一种选择其它数据使得PaToH划分器的划分结果并非最优。
- 2) 在验明算法的可行性和有效性后，将算法合并，并用递归策略完成多次优化操作。
- 3) 稍微修改C++代码后，在存储空间更大的计算机上对node数更多的数据进行划分任务。

4.2. 未来展望

FPGA电路正从可编程逻辑器件转变为异构融合、动态自适应的通用算力基座。在架构层面，多晶粒集成和三维堆叠将单器件逻辑容量增加到数亿门级，同时高速芯片间的互连和统一编程模型将多芯片FPGA阵列虚拟为单一巨型加速器。动态部分可重构能力将得到提升，通过芯片上集成低延迟重构控制器和智能划分处理器，FPGA能够在短时间内按需重组成不同的硬件管线，实时响应负载变化。设计工具方面人工智能驱动的自动优化将贯穿综合、划分、布局和布线全流程，图划分技术被深度嵌入编译栈，自主搜索算子切分、数据流架构与物理资源的最优映射，使开发门槛大幅降低。面向数据中心、自动驾驶和AI Infra等领域，FPGA将作为智能网卡、近存储计算和领域特定加速器的核心，与GPU、CPU构成深度耦合的计算单元，为自适应和高能效计算持续开拓边界。

作为基于划分的FPGA布局技术的理论基础，图划分技术正从静态、单目标优化向智能化、动态化和极限规模处理全面推进。未来，基于图神经网络和强化学习的划分算法将能够自主学习电路网表、社交网络等超大规模图的结构特征，在划分初期就预判关键路径和拥塞热点，生成最优解。面对数亿边的图，分布式流式划分框架将允许在不加载全图的情况下进行在线划分，支撑动态图数据的实时处理。三维集成与Chiplet时代的到来，将平面图划分问题推向多层三维空间的图划分问题。此外，图划分将与存算一体、近存计算等新架构深度融合，在划分阶段就协同规划数据放置与计算调度，突破“存储墙”瓶颈。图划分将不再只是离线预处理，而是贯穿数据流处理、芯片设计与科学计算全生命周期的核心技术。

正如1.1中所言，对与FPGA相关的研发、设计技术的需求与日俱增。未来十年，FPGA行业将在高速增长中实现价值跃迁。随着架构持续融合、工艺制程及工具链逐步突破，FPGA将深度融入AI与通信技术的核心脉络；同时，其独特定位与构建生态的强大能力，也将为整个智能计算领域注入更坚实的创新动能。而作为与FPGA相关的研发、设计技术的一环，多芯片FPGA电路的划分技术也会在未来不断发展，朝着更智能、更自动化、更系统级的优化的方向不断更新迭代。

附件

1. 原始数据文件实例
2. C++程序
3. 实践实例

参考文献

- [1] 田春生, 陈雷, 王源, 等. 面向FPGA的布局与布线技术研究综述[J]. 电子学报, 2022, 50(05): 1243-1254.
- [2] 黄宏赟, 王子健, 惠锋, 等. 一种多维约束感知的多裸片FPGA网表划分方法[J/OL]. 微电子学与计算机, 1-6[2026-04-15]. <https://link.cnki.net/urlid/61.1123.tn.20260319.1313.016>.
- [3] TRIMBERGER S M. Three ages of FPGAs: A retrospective on the first thirty years of FPGA technology [J]. Proceedings of the IEEE, 2015, 103(3): 318331.
- [4] TRIMBERGER S M. Three ages of FPGAs: a retrospective on the first thirty years of FPGA technology: this paper reflects on how Moore's law has driven the design of FPGAs through three epochs: the age of invention, the age of expansion, and the age of accumulation [J]. IEEE Solid-State Circuits Magazine, 2018, 10(2): 1629.
- [5] KHATKHATE A, LI C, AGNIHOTRI A R, et al. Recursive bisection based mixed block placement [J]. IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems, 2005, 24(5): 748761.
- [6] BUSTANY I, GASPRAYAN G, KAHNG A B, et al. An open-source constraints-driven general partitioning multi-tool for VLSI physical design [C] // IEEE/ACM International Conference on Computer-Aided Design (ICCAD). San Francisco, CA, USA. 2023: 1-9. DOI: 10.1109/ICCAD57390.2023.10323975.
- [7] ÇATALYÜREKÜ, DEVINE K, FARAJ M, et al. More recent advances in(hyper)graph partitioning [J]. ACM Computing Surveys, 2023, 55(12): 1-38. DOI: 10.1145/3571808.
- [8] 路文军, 刘红卫, 曹欢欢. 改进的宽度优先超图划分生成方法 [J]. 哈尔滨师范大学自然科学学报, 2025, 41(03): 5-10.
- [9] 李本正. 多FPGA系统结构感知的电路快速划分方法研究 [D]. 西安电子科技大学, 2023. DOI: 10.27389/d.cnki.gxadu.2023.004275.
- [10] 马永刚. 图划分方法及其在分布式网络环境下的应用 [D]. 大连理工大学, 2012.
- [11] 尚俊霖, 张振宇, 屈稳稳, 等. 面向分布式图计算的图划分技术综述 [J]. 计算机研究与发展, 2025, 62(01): 90-103.
- [12] 李金吉, 张岩峰, 巩树凤, 等. 流式处理的异步图处理框架 [J]. 软件学报, 2018, 29(03): 528-544. DOI: 10.13328/j.cnki.jos.005441.
- [13] 李贺, 刘延娜, 袁航, 等. 动态图划分算法研究综述 [J]. 软件学报, 2023, 34(02): 539-564. DOI: 10.13328/j.cnki.jos.006705.
- [14] Fischer E, Matsliah A, Shapira A. Approximate hypergraph partitioning and applications [J]. SIAM Journal on Computing, 2020, 36(7): 3155-3185.
- [15] Buluç A, Meyerhenke H, Safro I et al. Recent advances of graph partitioning [M]. Berlin: Springer International Publishing, 2016.
- [16] Demir E, Aykanat C, Cambazoglu B B. Clustering spatial networks for aggregate query processing: A hypergraph approach [J]. Information Systems, 2008, 33(1): 1-17.

- [17]Catalyurek U, Aykanat C.PaToH: Partitioning Tool for Hypergraphs(Manual) [CP/OL]. [2026-05-07]. <https://faculty.cc.gatech.edu/~umit/PaToH/manual.pdf>.
- [18]NIGHTINGALE A. Network-on-chip interconnect topologies explained[EB/OL]. (2023-08-03) [2026-05-07]. <https://www.ednasia.com/network-on-chip-interconnect-topologies-explained/>
- [19]IMANI M, ZOU Z W, BOSCH S, et al. Revisiting hyperdimensional learning for FPGA and low-power architectures[C]. 2021 IEEE International Symposium on High-Performance Computer Architecture (HPCA). South Korea: IEEE, 2021: 221234.
- [20]ZHANG J X, YANG T, LI Q Z, et al. An FPGA-based neural network overlay for ADAS supporting multi-model and multi-mode[C]. 2021 IEEE International Symposium on Circuits and Systems (ISCAS). South Korea: IEEE, 2021: 15.
- [21]KAN H W, LI R G, SU D D, et al. Trusted edge cloud computing mechanism based on FPGA cluster[C]. 2020 IEEE 8th International Conference on Computer Science and Network Technology (ICCSNT). China: IEEE, 2020. 146149.
- [22]刘公绪, 史凌峰, 辛东金. 基于FPGA 的零误差大数阶乘算法的设计与实现[J]. 电子学报, 2019, 47(5): 11801184.
- [23]蹇强, 张培勇, 王雪洁. 一种可配置的CNN 协加速器的FPGA实现方法[J]. 电子学报, 2019, 47(7):15251531.
- [24]SADEGHI A, LIGHVAN M Z, PRINETTO P. Automatic and simultaneous floorplanning and placement in field-programmable gate arrays with dynamic partial reconfiguration based on genetic algorithm[J]. Canadian Journal of Electrical and Computer Engineering, 2020, 43(4):224234.
- [25]HERATH K, PRAKASH A, JIANG G Y, et al. Ant Colony Optimization based module footprint selection and placement for lowering power in large FPGA designs[C]. 2018 International Conference on ReConfigurable Computing and FPGAs (ReConFig). Mexico: IEEE, 2018: 18.
- [26]SARI A, PSARAKIS M. Scrubbing-aware placement for reliable FPGA systems[J]. IEEE Transactions on Emerging Topics in Computing, 2020, 8(3): 564576.
- [27]STEINBRUNN M, MOERKOTTE G, KEMPER A. Heuristic and randomized optimization for the join ordering problem[J]. The VLDB Journal, 1997, 6: 191208.
- [28]CARL S. VLSI Placement and Global Routing Using Simulated Annealing[M]. Boston, MA, USA: Springer Science & Business Media, 2012.
- [29]GOPALAKRISHNAN P, LI X, PILEGGI L. Architecture-aware FPGA placement using metric embedding[C]. 2006 43rd ACM/IEEE Design Automation Conference. USA: IEEE, 2006: 460465.
- [30]LIN T H, BANERJEE P, CHANG Y W. An efficient and effective analytical place for FPGAs[C]. 2013 50th ACM/EDAC/IEEE Design Automation Conference (DAC). USA: IEEE, 2014: 16.
- [31]KIM, M C, LEE D J, MARKOV I L. SimPL: an effective placement algorithm[J]. IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems, 2012, 31(1):5060.

附录1

C++源代码如下:

```
class cell
{
public:
    std::string name;//the name of the cell
    std::string output_pin[360];//the name of the output pin of the cell
};

class node
{
public:
    std::string name;//the name of the node
    int cell;//the index of the cell
    int net;//the index of the net
    int die;//the index of the die
};

class net
{
public:
    std::string name;//the name of the net
};

class die
{
public:
    int capacity = 0;
    int list[5000];//list of nodes' index
    void add_to_die(int a);//add node to die
    void del_from_die(int a);//delet node from die
};

void die::add_to_die(int a)
{
    capacity++;
    if (capacity > 5000)
    {
```

```

        std::cout << "Die's capacity is overflow!" << std::endl;
    }
    else
    {
        list[capacity - 1] = a;
    }
}

void die::del_from_die(int a)
{
    int j;//a's index
    for (int i = 0; i < capacity; i++)
    {
        if (list[i] == a)
        {
            j = i;
            break;
        }
    }
    for (int i = j; i < capacity; i++)
    {
        list[i] = list[i + 1];
    }
    capacity--;
}

void init(string file_name, die *die_list, node *node_list, cell *cell_list, int &n_cells, int
&n_nets, int &n_pins, int &n_nodes)
//input: the name of file;
        //output: 4 dies, 10000 nodes, the number of cells, nets and pins in the file, the net list
    {
        int i_cell = -1, j_opin = 0;
        int i_pin = 0;
        int i_net = -1;
        n_cells = 0;
        n_nets = 0;
    }

```

```

n_pins = 0;
n_nodes = 0;
//initialize or clear the node list
for (int i = 0; i < 10000; i++)
{
node_list[i].name = "";
node_list[i].cell = -1;
node_list[i].net = -1;
node_list[i].die = -1;
}
//initialize cell list this research needs
cell_list[0].name = "LUT1";
cell_list[1].name = "LUT2";
cell_list[2].name = "LUT3";
cell_list[3].name = "LUT4";
cell_list[4].name = "LUT5";
cell_list[5].name = "LUT6";
cell_list[6].name = "FDRE";
cell_list[7].name = "CARRY8";
cell_list[8].name = "DSP48E2";
cell_list[9].name = "RAMB36E2";
cell_list[10].name = "IBUF";
cell_list[11].name = "OBUF";
cell_list[12].name = "BUFGCE";
n_cells = 13;
//open the files
ifstream file1(file_name + ".lib");
ifstream file2(file_name + ".nets");
ifstream file3(file_name + ".nodes");
ifstream file4(file_name + ".scl");
//check if the files are opened successfully
if(!file1.is_open())
{
    cout << "Cannot open file: " << file_name << endl;

```

```

return;
}
else if(!file2.is_open())
{
    cout << "Cannot open file: " << file_name << endl;
return;
}
else if(!file3.is_open())
{
    cout << "Cannot open file: " << file_name << endl;
return;
}
else if(!file4.is_open())
{
    cout << "Cannot open file: " << file_name << endl;
return;
}
string line;
//complete the cell list based on the .lib file
while (getline(file1, line))
{
    if (line.empty()) continue;
    istringstream iss(line);
    string token1, token2, token3;
    if (!(iss >> token1)) continue;
    if (!(iss >> token2)) continue;
    if (!(iss >> token3)) continue;
    if (token1 == "CELL")
    {
        for (int r = 0; r < n_cells; r++)
        {
            if (cell_list[r].name == token2)
            {
                i_cell = r;

```



```

break;
}
}
continue;
}
if (token1 == "END")
{
i_cell = -1;
j_opin = 0;
continue;
}
if (token3 == "OUTPUT" && i_cell != -1)
{
cell_list[i_cell].output_pin[j_opin] = token2;
j_opin++;
}
}
file1.close();
file4.close();
//initialize the node list and the cell list based on the .nodes file
while (getline(file3, line))
{
if (line.empty()) continue;
istringstream iss(line);
string token1, token2;
if (!(iss >> token1)) continue;
if (!(iss >> token2)) continue;
//write down the name of the node
node_list[i_pin].name = token1;
//find and write down the cell index of the node
int found_cell = -1;
if (!token2.empty())
{
for (int r = 0; r < n_cells; ++r)

```

```

{
if (cell_list[r].name == token2)
{
found_cell = r;
break;
}
}
}

node_list[i_pin].cell = found_cell;
n_nodes++;
++i_pin;
if (i_pin > 10000) break;
}

file3.close();

//initialize the node list and based on the .nets file
while (getline(file2, line))
{
if (line.empty()) continue;
istringstream iss(line);
string token1, token2;
if (!(iss >> token1)) continue;
if (!(iss >> token2)) continue;
if (line == "endnet") continue;
if (token1 == "net")
{ i_net++; n_nets++; continue; }

//write down the net index of the node
n_pins++;

int found_pin = -1;
for (int r = 0; r < n_pins; ++r)
{
if (node_list[r].name == token1)
{
found_pin = r;
break;
}
}
}

```

```

}
}

node_list[found_pin].net = i_net;
if (i_net > 10000) break;
}

file2.close();
}

void create_hgr(string file_name, node *node_list, int n_nodes, int n_nets, int n_pins)//create a
.hgr file for partitioning
{
ofstream file;
file.open(file_name + ".hgr", ios::out);
ifstream File(file_name + ".nets");
if (!File.is_open())
{
cout << "Cannot open nodes file: " << file_name + ".nets" << endl;
return;
}

file << "0" << " " << n_nodes << " " << n_nets << " " << n_pins << endl;
string line;
while (getline(File, line))
{
if (line.empty()) continue;
istringstream iss(line);
string token1;
if (!(iss >> token1)) continue;
if (token1 == "net") continue;
if (line == "endnet") { file << endl; continue; }
int found_pin = -1;
for (int r = 0; r < n_pins; ++r)
{
if (node_list[r].name == token1)
{
found_pin = r;

```

```

break;
}
}
file << found_pin << " ";
}
File.close();
file.close();
}

void place(string f_name, int n_dies, node *node_list)
{
//clear the die information in the node list
for (int r = 0; r < 10000; r++)
{
node_list[r].die = -1;
}

ifstream file(f_name + ".hgr.part." + to_string(n_dies));
if (!file.is_open())
{
cout << "Cannot open nodes file: " << f_name + ".hgr.part." +
to_string(n_dies) << endl;
return;
}

string line;
int i_node = 0;
while (getline(file, line))
{
if (line.empty()) continue;
istringstream iss(line);
string d;
while (iss >> d)
{
node_list[i_node].die = stoi(d);
i_node++;
}
}

```

```

}
file.close();
}

void check(string file_name, cell *cell_list, node *node_list, die *die_list, int &bridge, float
&average_cell_using)

    //caculate the number of bridges and the average cell using of the partitioning result {
bridge = 0;
ifstream file1(file_name + ".nets");
if (!file1.is_open())
{
    cout << "Cannot open file: " << file_name + ".nets" << endl;
    return;
}
//caculate the average cell using
int c_M = 0;
int c_m = 5000;
for (int i = 0; i < 4; i++)
{
    if (die_list[i].capacity > c_M) c_M = die_list[i].capacity;
    if (die_list[i].capacity < c_m) c_m = die_list[i].capacity;
}
average_cell_using = (c_M - c_m) * 1.0 / c_m;
//caculate the SLL using
int d_M = 0, d_m = 3;
string line;
while (getline(file1, line))
{
    if (line.empty()) continue;
    stringstream iss(line);
    string token;
    if (!(iss >> token)) continue;
    if (token == "net") continue;
    if (token == "endnet")
    {

```

```

bridge += (d_M - d_m);
d_M = 0;
d_m = 3;
continue;
}

int found_node = -1;
for (int r = 0; r < 10000; r++)
{
if (node_list[r].name == token)
{
found_node = r;
break;
}
}

if (node_list[found_node].die > d_M) d_M = node_list[found_node].die;
if (node_list[found_node].die < d_m) d_m = node_list[found_node].die;
}

file1.close();
}

void sort_die(die *die_list, int *d_sorting, int n_dies)
{
int i_d_c_m = -1, d_c_m = 5000;
int is_sorted[10];

for (int i = 0; i < n_dies; i++) is_sorted[i] = 0;
for (int i = n_dies; i > 0; i--)
{
for (int j = 0; j < n_dies; j++)
{
if (die_list[j].capacity < d_c_m && is_sorted[j] == 0)
{
i_d_c_m = j;

d_c_m = die_list[j].capacity;
}
}
}
}

```

```

}
d_sorting[i - 1] = i_d_c_m;
is_sorted[i_d_c_m] = 1;
}
}

void update_die_list(node *node_list, die *die_list, int n_dies, int n_nodes)
{
//clear the die list
for (int i = 0; i < n_dies; i++)
{
for (int j = 0; j < die_list[i].capacity; j++)
{
die_list[i].list[j] = -1;
}
die_list[i].capacity = 0;
}
for (int i = 0; i < n_nodes; i++)
{
die_list[node_list[i].die].add_to_die(i);
}
}

void optimize(string file_name, cell *cell_list, die *die_list, node *node_list, int n_dies, int
n_nets, int n_nodes, float average_cell_using, int bridge)
{
ifstream file(file_name + ".nets");
if (!file.is_open())
{
cout << "Cannot open file: " << file_name + ".nets" << endl;
return;
}
file.close();
//select the optimization mode
int mode = -1;
float a_c_u;

```

```

int b;

    check(file_name, cell_list, node_list, die_list, b, a_c_u);
if (b > bridge) mode = 1;
else
{
    if (a_c_u > average_cell_using) mode = 2;
    else mode = 0;
}

if (mode == -1) mode = 0;
//optimization
switch (mode)
{
    case 1://optimize the partitioning result based on the limited SLL
    {
        //find the net with the largest SLL using
        int d_M = 0, d_m = 3, i_net = 0;
        int n_b_M = 0;
        int i_n_b_M = -1;
        int i_node = 0, n_c_M = 0;
        string line;
        file.open(file_name + ".nets");
        while (getline(file, line))
        {
            if (line.empty()) continue;
            stringstream iss(line);
            string token;
            if (!(iss >> token)) continue;
            if (token == "net") continue;
            if (token == "endnet")
            {
                if (d_M - d_m > n_b_M)
                {
                    if (n_c_M < i_node)
                    {

```



```

n_b_M = d_M - d_m;
i_n_b_M = i_net;
n_c_M = i_node;
}
}
i_net++;
i_node = 0;
d_M = 0;
d_m = 3;
continue;
}
int found_node = -1;
for (int r = 0; r < n_nodes; r++)
{
if (node_list[r].name == token)
{
found_node = r;
break;
}
}
if (node_list[found_node].die > d_M) d_M = node_list[found_node].die;if (node_list[found_node].die <
d_m) d_m = node_list[found_node].die;i_node++;
}
file.close();

if (n_b_M == 0)//if the SLL using of every net is 0, then the partitioning result can't be
optimized any more
{
cout << "Can't optimize the partitioning result any more. " << endl;
return;
}
else
{
//move the nodes
file.open(file_name + ".nets");

```

```

d_M = 0, d_m = 3, i_net = 0;
while (getline(file, line))
{
    if (line.empty()) continue;
    stringstream iss(line);
    string token;
    if (!(iss >> token)) continue;
    if (token == "net") continue;
    if (token == "endnet")
    {
        if (i_net == i_n_b_M) break;
        else
        {
            i_net++;
            continue;
        }
    }
    if (i_net == i_n_b_M)
    {
        int found_node = -1;
        for (int r = 0; r < n_nodes; r++)
        {
            if (node_list[r].name == token)
            {
                found_node = r;
                break;
            }
        }
        if (node_list[found_node].die > d_M) d_M =
            node_list[found_node].die;
        if (node_list[found_node].die < d_m) d_m =
            node_list[found_node].die;
    }
    else continue;
}

```

```

}

file.close();

file.open(file_name + ".nets");

i_net = 0;

while (getline(file, line))
{
    if (line.empty()) continue;
    stringstream iss(line);
    string token;
    if (!(iss >> token)) continue;
    if (token == "net") continue;
    if (token == "endnet")
    {
        if (i_net == i_n_b_M) break;
        else
        {
            i_net++;
            continue;
        }
    }

    if (i_net == i_n_b_M)
    {
        int found_node = -1;
        for (int r = 0; r < n_nodes; r++)
        {
            if (node_list[r].name == token)
            {
                found_node = r;
                break;
            }
        }

        if (node_list[found_node].die == d_M)
            node_list[found_node].die = d_M - 2;
        else continue;
    }
}

```

```

}
else continue;
}
file.close();
}

    update_die_list(node_list, die_list, n_dies, n_nodes);
break;
}

    case 2://optimize the partitioning result based on the limited average cell using {
int d_sort[4];
sort_die(die_list, d_sort, n_dies);
int i_net = 0, i_node = 0, i_net_selected = -1;
int has_die[10], is_in_die[10];
int net_capacity_M = 0;
int S = -1;
for (int i = 0; i < n_dies; i++)
{
has_die[i] = 0;
is_in_die[i] = 0;
}
int sum_is_in_die;
string line;

    //select a net to move its nodes (or the whole net)
while (getline(file, line))
{
if (line.empty()) continue;
stringstream iss(line);
string token;
if (!(iss >> token)) continue;
if (token == "net") continue;
if (token == "endnet")
{
for (int i = 0; i < n_dies; i++) sum_is_in_die += is_in_die[i];
if (is_in_die[d_sort[0]] == 0)

```

```

{
i_net++;
i_node = 0;
for (int i = 0; i < n_dies; i++)
{
has_die[i] = 0;
is_in_die[i] = 0;
}
sum_is_in_die = 0;
continue;
}
else
{
if (sum_is_in_die == 1)
{
i_net++;
i_node = 0;

for (int i = 0; i < n_dies; i++)
{
has_die[i] = 0;
is_in_die[i] = 0;
}
sum_is_in_die = 0;
continue;
}
else
{
if (i_node > net_capacity_M)
{
net_capacity_M = i_node;
i_net_selected = i_net;
S = 0;
}
i_net++;

```

```

i_node = 0;
for (int i = 0; i < n_dies; i++)
{
has_die[i] = 0;
is_in_die[i] = 0;
}

sum_is_in_die = 0;
continue;
}
}
}

int found_node = -1;
for (int r = 0; r < n_nodes; r++)
{
if (node_list[r].name == token)
{
found_node = r;
break;
}
}

is_in_die[node_list[found_node].die] = 1;
has_die[node_list[found_node].die]++;
}

file.close();

if (i_net_selected == -1)//special situation
{
S = 1;
i_net = 0;
i_node = 0;
for (int i = 0; i < n_dies; i++)
{
has_die[i] = 0;
is_in_die[i] = 0;
}
}

```

```

file.open(file_name + ".nets");
while (getline(file, line))
{
    if (line.empty()) continue;
    stringstream iss(line);
    string token;
    if (!(iss >> token)) continue;
    if (token == "net") continue;
    if (token == "endnet")
    {
        if (is_in_die[d_sort[0]] == 1)
        {
            i_net_selected = i_net;
            break;
        }
        else
        {
            i_net++;
            i_node = 0;
            for (int i = 0; i < n_dies; i++)
            {
                has_die[i] = 0;
                is_in_die[i] = 0;
            }
            continue;
        }
    }
    int found_node = -1;
    for (int r = 0; r < n_nodes; r++)
    {
        if (node_list[r].name == token)
        {
            found_node = r;
            break;
        }
    }
}

```

```

}
}

is_in_die[node_list[found_node].die] = 1;
}

file.close();

        }
        if (S == -1)

{
cout << "Can't optimize the partitioning result any more. " << endl;
return;
}
//move nodes (or net)
i_net = 0;
i_node = 0;
for (int i = 0; i < n_dies; i++)
{
has_die[i] = 0;
is_in_die[i] = 0;
}
file.open(file_name + ".nets");
while (getline(file, line))
{
if (line.empty()) continue;
stringstream iss(line);
string token;
if (!(iss >> token)) continue;
if (token == "net") continue;
if (token == "endnet")
{
if (i_net != i_net_selected)
{
i_net++;
continue;
}
}
}

```



```

else break;
}

if (i_net != i_net_selected) continue;
else
{
    int found_node = -1;
    for (int r = 0; r < n_nodes; r++)
    {
        if (node_list[r].name == token)
        {
            found_node = r;
            break;
        }
    }

    is_in_die[node_list[found_node].die] = 1;
    has_die[node_list[found_node].die]++;
}

}

file.close();

i_net = 0;

file.open(file_name + ".nets");
while (getline(file, line))
{
    if (line.empty()) continue;
    stringstream iss(line);
    string token;
    if (!(iss >> token)) continue;
    if (token == "net") continue;
    if (token == "endnet")
    {
        if (i_net != i_net_selected)
        {
            i_net++;
            continue;
        }
    }
}

```

```

}
else break;
}
if (i_net != i_net_selected) continue;
else
{
int found_node = -1;
for (int r = 0; r < n_nodes; r++)
{
if (node_list[r].name == token)
{
found_node = r;
break;
}
}

if (node_list[found_node].die != d_sort[0]) continue;
else
{
if (S == 1) node_list[found_node].die = d_sort[3];
else
{
if (is_in_die[d_sort[3]] == 1) node_list[found_node].die =
d_sort[3];
else if (is_in_die[d_sort[2]] == 1)

node_list[found_node].die = d_sort[2];
else node_list[found_node].die = d_sort[1];
}
}
}
}

file.close();

update_die_list(node_list, die_list, n_dies, n_nodes);
break;

```

```

}
default:
return;
}
return;
}

void create_result_file(string file_name, node *node_list, int n_nodes, int n_dies)
{
ofstream file;
    file.open(file_name + ".hgr.part." + to_string(n_dies) + ".optimized");
if (!file.is_open())
{
cout << "Cannot open file: " << file_name + ".hgr.part." + to_string(n_dies) + ".optimized" << endl;
return;
}
for (int i = 0; i < n_nodes; i++)
{
file << node_list[i].die << endl;
}
file.close();
}

int main()
{
int m_s;
//mode selection:
    //0 for preparation, 1 for partitioning, 2 for placement, 3 for checking, 4 for optimization
string file_name;
die dies[4];
node nodes[10000];
cell cells[13];
int n_c, n_n, n_p, n_no;
int bridge, _b;
float a_cell_using, _a_c_u;
cin >> file_name;

```

```

    init(file_name, dies, nodes, cells, n_c, n_n, n_p, n_no);
while (true)
{
    cout << "Please select the mode:\n0 for preparation, 1 for partitioning, 2 for placement, 3 for
checking, 4 for optimization, others for exit. " << endl;
    cin >> m_s;
    switch (m_s)
    {
    case 0:
        create_hgr(file_name, nodes, n_no, n_n, n_p);
        break;
    case 1:
        //part(file_name + ".hgr", 4); /*using PaToH*/
        break;
    case 2:
        place(file_name, 4, nodes);
        for (int r = 0; r < n_no; r++)
            dies[nodes[r].die].add_to_die(r);
        break;
    case 3:
        check(file_name, cells, nodes, dies, bridge, a_cell_using);
        cout << "The average cell using is: " << a_cell_using << endl;
        cout << "The SLL using is: " << bridge << endl;
        break;
    case 4:
        cout << "Please input the requested average cell using and SLL: " << endl;
        cin >> _a_c_u >> _b;
        optimize(file_name, cells, dies, nodes, 4, n_n, n_no, _a_c_u, _b);
        create_result_file(file_name, nodes, n_no, 4);
        break;
    default:
        return 0;
    }
}
}

```

}

致谢

首先，感谢郭龙坤老师对我耐心且专业的指导，他在论文写作流程和方法方面对我的教导使我受益匪浅，从而让我顺利完成本次毕业设计（论文）。我认为一篇论文不能代表我在专业方面的水平，更不应该止步于此，而是要学习郭龙坤老师不断学习的精神和勤奋求真的治学态度，在相关领域开拓进取，学以致用，成为该领域的人才，不负郭龙坤老师的谆谆教诲。

其次，感谢给我帮助的所有同学以及学长学姐，他们不仅为我解答了很多我不熟悉的难题，也为我送来相关资料和论文写作技巧，让我体会到同学之间互帮互助、团结奋进的真挚友情，这种温暖、团结、进取的学习气氛，让我感动，让我一生铭记。

最后感谢我的家人，是他们多年来对我学业的支持才让我走到这一步，才使我得以顺利完成学业。

报告指标说明：

- 1.全文总相似比 = 复写率 + 自引率 + 他引率 + 专业术语。
- 2.复写率：指相似或疑似重复内容在全文中的比重。
- 3.自引率：指引用本人发表内容占全文的比重，需正确标注引用。
- 4.他引率：指引用他人内容占全文的比重，需正确标注引用。
- 5.专业术语率：指公式定理、法律条文、行业用语等在全文中的比重。
- 6.去除引用本人文献相似率：指去除本人发表部分后，相似或引用内容占全文的比重，需正确标注引用。
- 7.去除专业术语相似率：指去除专业术语后，相似或引用内容占全文的比重。
- 8.自写率：指原创内容在全文中的比重。
- 9.典型相似文章：指相似或引用内容占全文总相似比超过30%的文章。

总片段数量 = 相似片段 + 引用片段。相似片段中“综合”包括：《中文主要报纸全文数据库》《中国专利特色数据库》《中国主要会议论文特色数据库》《港澳台文献资源》《图书资源》《维普优先出版论文全文数据库》《年鉴资源》《古籍文献资源》《IPUB原创作品》。

须知：

- 报告编号系送检论文检测报告在本系统中的唯一编号。
- 本报告为维普论文检测系统算法自动生成，仅对您所选择比对资源范围内检验结果负责，仅供参考。



微信公众号

福州大学